

Report:

1. Connection Pool

In the original version:

- A new connection was created each time without being reused.
- There was no upper limit to the number of connections.
- There was no blocking behavior when connections were full.
- The ConnectionManager was a simple and insecure object.

Solution using Connection Pool:

The Connection Pool pattern was implemented as follows:

Create the ConnectionPool interface:

```
package connections;

public interface ConnectionPool {

    Connection acquire() throws
        InterruptedException;

    void release(Connection connection);
```

```
}
```

Create a SimpleConnectionPool:

```
package connections;

import
java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;

public class SimpleConnectionPool implements
ConnectionPool {

    private final BlockingQueue<Connection>
pool;

    // DEFAULT constructor (fixes the red
error)

    public SimpleConnectionPool() {
        this(10); // default pool size
    }
}
```

```
// custom constructor

public SimpleConnectionPool(int maxSize) {

    this.pool = new
ArrayBlockingQueue<>(maxSize);

    for (int i = 1; i <= maxSize; i++) {

        pool.offer(new Connection("Conn-"
+ i));

    }

}

@Override

public Connection acquire() throws
InterruptedException {

    return pool.take(); // waits if empty

}

@Override

public void release(Connection connection)
{

    if (connection != null) {
```

```
        pool.offer(connection);  
    }  
}  
}
```

A real pool is implemented using BlockingQueue:

- It allows a maximum of (10) connections.
- It returns a ready connection on request.
- It waits for (blocks) if a connection is unavailable.

Using the Pool Inside the Proxy

JobExecutor is no longer responsible for establishing connections.

Now:

- The proxy receives the connection from the pool
- Then it passes it to JobExecutor
- And finally it returns it to the pool

2. Prototype Pattern

The problem with the old system:

TemplateManager was rebuilding each template from scratch using:
//Thread.sleep(3000)

This resulted in:

- Extreme slowness
- Unnecessary redundancy
- Lack of scalability

To solve using Prototype Pattern:
Create a JobPrototype interface

```
package templates
```

```
import job.Job;

public interface JobPrototype {
    Job cloneJob();
}
```

Creating real templates for each job type

-EmailJobTemplate

-DataJobTemplate

-ReportJobTemplate

Each template contains:

//name

//config

//type

//method :cloneJob()

Ex:

```
public Job cloneJob() {
    String id = type + "-" + System.currentTimeMillis();
    return new Job(id, type, name, config);
}
```

Building a registry of templates:

```
JobTemplateRegistry registry = new JobTemplateRegistry();
```

Using a clone instead of rebuilding:

```
Job reportJob = registry.get("REPORT").cloneJob();
```

3.Strategy Pattern:

The problem with the old system:

JobExecutor relied on if/else.

This is contrary to the OCP principle.

The solution using Strategy Pattern

Creating a Job Execution Strategy Interface:

```
package executor;

import job.Job;

public interface JobExecutionStrategy {
    void execute(Job job);
}
```

Creating independent strategies:

-EmailJobStrategy

-DataJobStrategy

-ReportJobStrategy

Creating a Strategy Factory to Defining the Appropriate Strategy:

```
JobExecutionStrategy strategy = factory.getStrategy(job.getType());
```

JobExecutor update to become:

```
JobExecutionStrategy strategy = factory.getStrategy(job.getType());
strategy.execute(job);
```

4.Proxy Pattern:

The problem with the old system:

JobExecutor used to do everything:

Execute the job

Manage connections

Verify permissions

Measure time

Print

-This violates SRP and makes the class very complex.

The solution using Proxy Pattern

It performs the following:

1. Verifying user permissions:

```
if (!job.getRequestedBy().hasPermission(job.getType())) {  
    throw new RuntimeException("User not allowed for this job type");  
}
```

2. Recording the execution time:

```
long start = System.currentTimeMillis();
```

3. Take a connection from ConnectionPool:

```
Connection conn = pool.acquire();
```

4. Passing the job to the actual JobExecutor

```
realExecutor.executeJob(job);
```

5. Reconnect to the Pool

```
pool.release(conn);
```

6. Print execution time

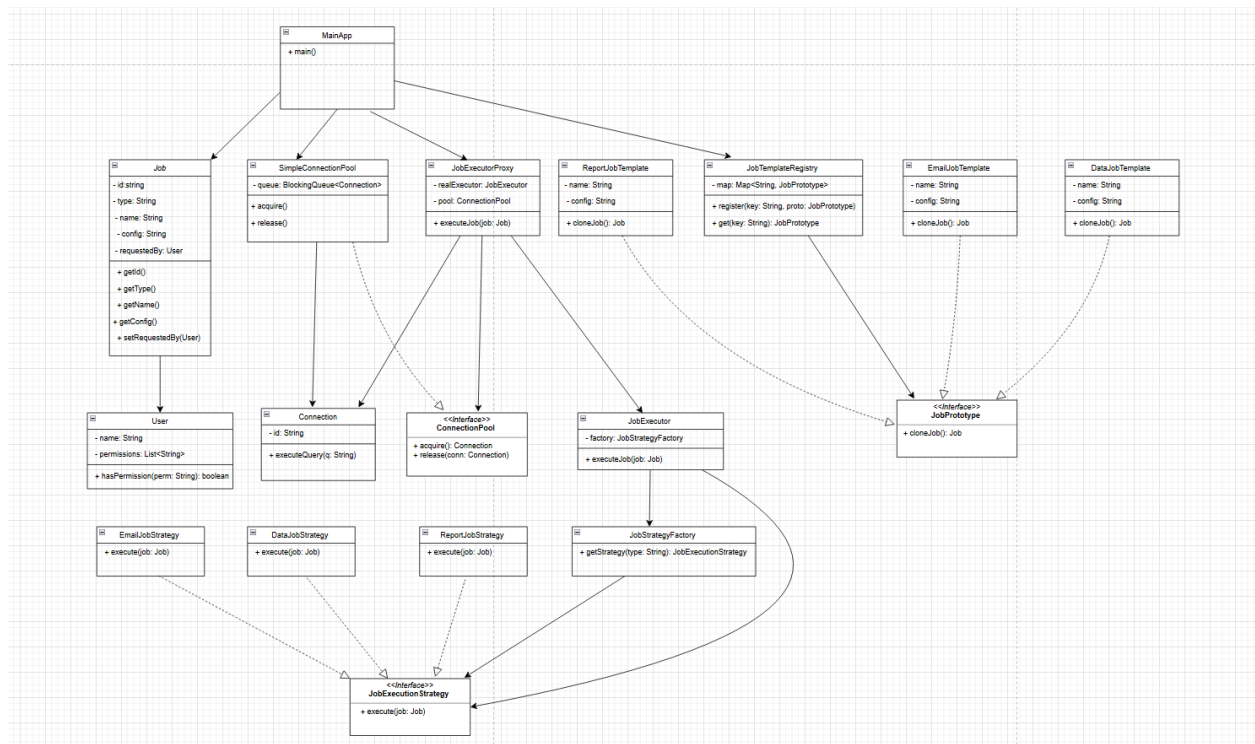
```
System.out.println("[Proxy] Execution time: " + duration + " ms");
```

The final output:

```
C:\Users\MASTAER\.jdk\openjdk-25\bin\java.exe "-javaagent:C:\Program Files\JetBrains\
[Proxy] Starting job: SalesMonthly
[ReportJob] Generating report: SalesMonthly
Config: format=PDF
[Proxy] Finished job: SalesMonthly
[Proxy] Execution time: 15 ms

Process finished with exit code 0
```

UML class diagram:



Design Decision Document

Describe:

- Why each design pattern was chosen
- Connection Pool Why did we choose it?

- Because creating a new connection for each job is expensive and slow.
- The Pool allows for connection reuse and reduces resource consumption.
- An alternative like creating a connection for each process is inefficient.

Prototype Why did we choose it?

- Building templates used to take 3 seconds each time.

- The Prototype allows for quickly copying a ready-made template without rebuilding it.
- Factory alone is insufficient because it would always rebuild the template.

Strategy Why did we choose it?

- To eliminate the long if/else function within the JobExecutor.
- Each Job type now has its own independent class.
- Adding a new Job type is easier without modifying the JobExecutor.

Proxy Why did we choose it?

To add:

- User permission verification
- Logging
- Execution time measurement
- Connection management from the Pool
Without modifying the core code.

- What alternatives you considered

Connection Pool

Alternative: Creating a new connection for each job

Slow , Restrictive , Not scalable.

Prototype

Alternative: TemplateFactory builds a template every time

Always incurs the same cost , Inefficient.

Strategy

Alternative: Switch-case or if/else

Violates OCP and makes JobExecutor not scalable.

Proxy

Alternative: Placing logging + authorization within JobExecutor

Causes overlapping responsibilities and breaks SRP.

- How the patterns interact

- MainApp creates templates using the Prototype Registry.
- The proxy receives the job and then:
 - 1.Verifies user permissions.
 - 2.Obtains a connection from the Connection Pool.
 - 3.Passes the job to the JobExecutor.
 - 4.The JobExecutor uses a strategy to determine the execution logic.

After execution:

- The proxy records the time.
- Returns the connection to the Pool.

The entire system utilizes all patterns in a seamless manner.

- How your architecture improves scalability, flexibility, and maintainability

Scalability

- Connection Pool allows for the execution of a large number of jobs without consuming resources.
- Prototype reduces job creation time when dealing with a large number of jobs.

Flexibility

- Strategy allows adding new job types without modifying existing code.
- Proxy allows adding new features without changing the execution logic.

Maintainability

- Segmented Responsibilities Reduction (SRP) across all components.
- Directed Delay (DRY).
- Easier unit testing for each component.

Issue Analysis Report

1) Issues in MainApp

The original code rebuilt templates from scratch each time, causing significant slowdowns and unnecessary redundancy.

It also relied directly on ConnectionManager, creating a strong dependency and making modifications difficult.

After the redesign, we used Prototype to avoid rebuilding and Connection Pool for better connection management.

2) Job Issues

The job type was stored as a single text file, which led to errors and made verification difficult.

Also, the config file was a single, unstructured piece of text, and the requestedBy file was changeable after the job was created.

After modification, the execution logic became separate using a strategy, and permission checks were performed via a proxy, not within the job itself.

3) ConnectionManager Issues

It created a new connection for each job, without a pool, which consumed resources and caused slowdowns.

It could also exceed the maximum connection limit because it didn't use proper connection management.

The solution was to create a SimpleConnectionPool that allowed for reusing connections and adjusting their number.

4) TemplateManager Issues

The code was very repetitive, and each template was created from scratch, resulting in poor performance.

Its responsibilities were numerous and unclear.

We used Prototype + Registry to provide ready-made versions of templates without duplication or high costs.

5) JobExecutor Issues

The code contained a large if/else statement to define the job type, which violates OCP principles and makes adding new types difficult.

It also handled everything: calling, executing, and printing, which violates SRP.

After the redesign, we used the Strategy Pattern to isolate the execution logic and a proxy to add permissions, time, and pool connections.

6) System Issues as a Whole

The old design was weak in terms of scalability and maintainability, and it had significant redundancy and a lack of responsibility separation.

The new design is more modular and scalable, and each part operates independently using the four Design Patterns.